Projektuppgift

tT Dev - en projektuppgift i Webbutveckling

***Författare**:* Mathias Thorgren
***Kurs**:* DT224G Introduktion till webbutveckling
***Datum**:* 2026-11-01

1 Innehållsförteckning

2 Sammanfattning 3

3 Introduktion 3

4 Teori 4

4.1 Grundläggande webbteknik 4

4.2 Responsiv webbdesign och användbarhet 4

4.3 Skalbar webbarkitektur och versionshantering 5

4.4 Inbäddning och resurshantering 5

5 Metod 6

5.1 Utvecklingsmiljö och verktyg 6

5.2 Initial design 6

5.3 Prototyp 6

5.4 Interaktivitet och JavaScript 6

5.5 Innehåll och inbäddning 7

5.6 Validering och korrektur 7

6 Genomförande 8

7 Resultat 9

8 Slutsatser 10

9 Källförteckning 11

2 Sammanfattning

3 Introduktion

För en utvecklare är det avgörande att kunna påvisa sin tekniska kompetens och sin förmåga att skapa välfungerande programvara. I stället för att enbart förlita sig på statiska meriter i skriven form kan det därför vara fördelaktigt att återge en inblick i avslutade projekt, och därmed ge en tydligare bild av ens erfarenheter.

Detta projekt går ut på att jag skall skapa en webbsida som presenterar mina utvecklingsprojekt på ett professionellt och tydligt vis, likt en interaktiv och dynamisk portfolio. Om projektet lyckas så kan webbsidan med fördel användas som ett komplement till mitt CV. Den huvudsakliga målgruppen för webbplatsen är potentiella uppdragsgivare eller rekryterare

För att stödja utvecklingsarbetet kommer två wireframes att tas fram. Dessa utgör grova skisser av webbplatsens utformning för såväl mindre (mobil) som större (dator) skärmar. Därefter produceras en mockup, vilket är en mer detaljerad representation som avser motsvara hur slutprodukten faktiskt kommer att se ut. Mockupen fungerar som en mall och vägledning för den slutliga designen och dess grafiska profil.

Vidare innefattar målbilden att produkten skall vara skalbar och användbar över tid, det vill säga att administrering och vidareutveckling kan fortlöpa efter projektets slut. Detta ställer krav på en modulär uppbyggnad. Funktionerna ska vara återanvändbara inom projektets och följa etablerad praxis för att stödja framtida integrationer med extern infrastruktur och externa tjänster.

4 Teori

4.1 Grundläggande webbteknik

Webbplatser bygger i regel på tre sammanverkande tekniker som tydligt skiljer på struktur, presentation och beteende [1]:

- HTML (HyperText Markup Language): Utgör webbplatsens semantiska struktur och innehållsarkitektur.
- CSS (Cascading Style Sheets): Hanterar den visuella utformningen, layouten och typografin.
- JavaScript: Ett skriptspråk som tillför dynamik, interaktivitet och klientlogik.

HTML-dokumentet fungerar som startpunkten för en webbsida. Standarden använder uppmärkning (*markup*) för att strukturera text, bilder och annat innehåll i webbläsaren med hjälp av element som omges av s.k. *markup-taggar* [1]. För att uppnå en god struktur och underhållbarhet delas koden upp i separata filer utifrån deras specifika ansvarsområden, snarare än att samlas i en enda fil.

HTML-standarden används uteslutande för att strukturera innehållet logiskt. Även om HTML tillåter viss formatering hanteras detta numera primärt via CSS. Genom CSS nås enskilda HTML-element eller grupper av element, vilket gör att den visuella utformningen kan appliceras med hög precision och granularitet. På samma sätt avskiljs webbplatsens interaktiva beteende och logik till JavaScript.

4.2 Responsiv webbdesign och användbarhet

Responsiv och tillgänglig webbdesign innebär att en webbplats anpassas med hänsyn till slutanvändarens förutsättningar. Centrala delar inom detta område är:

- *User Experience* (UX) och *User Interface* (UI): Principer för användarupplevelse, navigering och visuell hierarki som säkerställer en intuitiv upplevelse.
- Tillgänglighetsprinciper för webbutveckling.
- Iterativ design- och utvecklingsprocess: Ett strukturerat arbetssätt som rör sig från grova skisser (wireframes) till detaljerade mockups, prototyper och sluttester.
- Layout: Teknik som *CSS Grid* och *Flexbox* för att skapa strukturerade, flexibla och anpassningsbara gränssnitt.

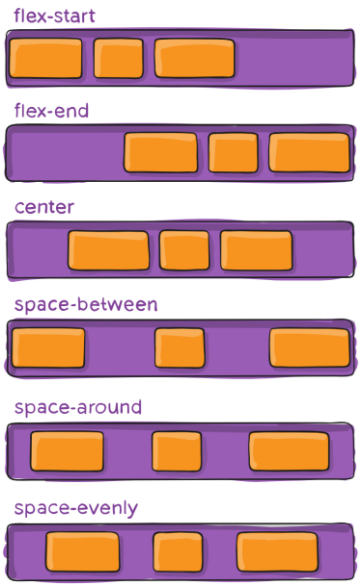
Tillgänglighet är en central aspekt av modern webbutveckling. *World Wide Web Consortium* (W3C) framhåller att webben i grunden är utformad för att vara tillgänglig för alla människor, oavsett hårdvara, mjukvara, språk, plats eller förutsättningar:

> *The Web is fundamentally designed to work for all people, whatever their hardware, software, language, location, or ability. When the web meets this goal, it is accessible to people with a diverse range of hearing, movement, sight, and cognitive ability.* [2]

Design för god *UI* -och *UX-design* omfattar designavgöranden avseende att förbättra mottagligheten av webbsidans innehåll [3]. Överväganden för UI är fokuserat på de visuella aspekterna, såsom samverkan av färger eller hur element placeras för användarnas synfält. UX däremot är inriktat på att möta och anpassa design efter användares faktiska förväntningar, vilket gör att detta även relaterar till

kontinuerlig testning. Genom att arbeta iterativt med allt från wireframes till interaktiva prototyper kan utvecklare testa idéer tidigt [4]. Denna utvecklingsmetod är fördelaktig för att kontinuerligt kunna mäta och följa upp hur produkten förhåller sig till exempelvis sina initiala designmål eller tillgänglighetskrav.

CSS Grid och Flexbox är tekniker som ger full kontroll över elementens placering och gör det möjligt att styra den visuella anpassningen på ett effektivt sätt i olika skärmstorlekar [5].



Figur 1 - Exempel på stilkonfiguration med flexbox: distribution av innehåll [5]

4.3 Skalbar webbarkitektur och versionshantering

För att en webbplats ska kunna underhållas och vidareutvecklas effektivt över tid krävs en skalbar arkitektur samt en kontrollerad utvecklingsprocess med spårbarhet. Detta uppnås genom:

- Modulär kodstruktur: Återanvändbara komponenter och funktioner.
- *Separation of Concerns* (SoC): Principen att hålla funktionalitet decentraliserad och modulär.
- Versionshantering: Användning av verktyg som *Git* för att spåra kodändringar, möjliggöra parallell utveckling och säkerställa en spårbar historik under projektets livscykel [2].

Med projektets målsättning att skapa en plattform för fortsatt underhåll av prototyper och projektpresentationer ökar behovet av flexibilitet. Dessa aspekter underlättar på olika vis i det avseendet. En tydlig separation av ansvarsområden (SoC) förbättrar systemets skalbarhet och läsbarhet, samtidigt som det underlättar kodens återanvändbarhet i nya sammanhang [6]. Genom att isolera olika funktioner minskar risken för att ändringar får oönskade effekter som sprids i koden. Kombinaterat med modulär struktur och versionshantering kan man därför skapa en mer kontrollerad miljö för fortlöpande utveckling.

4.4 Inbäddning och resurshantering

För att integrera externa moduler eller interaktiva applikationer krävs en lämplig inbäddningsteknik:

- Inline Frames (<iframe>): En metod för att isolera och bädda in en extern resurs direkt i den befintliga sidan, utan att påverka huvudsidans kodkontext eller stilregler.

I detta projekt avses inbäddning av både video, bilder och interaktiva prototyper. Fokus ligger primärt på videolänkar, men den valda tekniken skapar också en stabil grund för att hantera framtida och mer resurskrävande webbintegrationer [7].

5 Metod

Detta kapitel redogör för de tillvägagångssätt, verktyg och metoder som har tillämpats under planering, utveckling och testning av webbplatsen.

5.1 Utvecklingsmiljö och verktyg

Projektet har utvecklats med primär utgångspunkt i *Visual Studio Code* (VS Code) som primär integrerad utvecklingsmiljö (*Integrated Development Environment*, IDE). VS Code är det primära gränssnittet för utvecklingen, men för att uppnå spårbarhet och ge goda förutsättningar för underhåll så has versionshantering utförts kontinuerligt via Git. Utvecklingen kan även testas genom att filerna körs på en lokal webbserver, vilket är ett bra testverktyg som till stor del motsvarar hur produkten kommer att fungera online.

Via tjänsten GitHub har projektet synkroniserats i en struktur med separerade grenar (”main” och ”dev”). Först när en fil eller funktion är framtagen, testad och klar läggs den till i main.

Projektet initierades med förberedande av den lokala arbetskatalogen, med en initial synkronisering till huvudkatalogen i GitHub (repository).

5.2 Initial design

Arbetet påbörjades med inleddes med framtagande av wireframes. Dessa skissar syftar till att enkelt påvisa hur produkten skall se ut på både mindre vertikala (mobil) samt större horisontella (dator) skärmar. Wireframen informerar vissa snabba överväganden, men kort därefter påbörjades arbetet av att ta fram mockups som kommer att informera den kommande implementationen för interaktiv webbmiljö.

5.3 Prototyp

Den första prototypen skapades därefter med ett ”skelett” utav HTML-struktur och modulär stilanpassning i CSS. Dessa två komponenter ihop utgör en delvis interaktiv prototyp, då navigation mellan HTML-sidor (filer) blir möjligt på stilanpassade element. De tidiga versionerna av HTML- och CSS-filerna byggdes till fördel upp kvickt, då första iterationen omöjligen blir nära det tänkta resultat, oaktat hur mycket energi och tid som dedikeras. Härifrån utfördes konstanta iterationer utav både HTML och CSS fram till slutet av projektet.

5.4 Interaktivitet och JavaScript

När navigation och stil tycktes vara nästintill identisk med mockups var det dags att skapa de mer komplexa funktionerna, och de funktioner som är baserat i interaktivitet eller nätverkstjänster. Kontaktformuläret är ett bra exempel på en komponent som krävde utökad logik. Delvis behöver meddelandet som matas in skall nå en mottagare, men meddelandet behöver också valideras och felhanteras på ett intuitivt sätt för användaren.

Utöver logik kopplat till kontaktformuläret skapades även funktioner som avser förhöja användarupplevelsen, exempelvis att menyknappar kan minska när användaren navigerar vertikalt.

5.5 Innehåll och inbäddning

Då målprodukten är en (portfolio-) plattform och inte det faktiska innehållet (uppvisning av applikationer och prototyper), så var projektet inte heller särskilt fokuserad på just innehåll. Detta innebär att till stor del blev innehållet kortare beskrivande textstycken eller rubriker med tillfälliga bilder (placeholders). Likväl var det viktigt att säkerställa att varje potentiellt innehåll har en förberedd plats och infrastruktur. Därför skapades några tillfälliga komponenter, såsom förberedda ”projekt-kort”, HTML-mall för framtida projekt och även inbäddning av en tillfällig video med <iframe>.

5.6 Validering och korrektur

6 Genomförande

7 Resultat

8 Slutsatser

9 Källförteckning

[1] Mozilla Developer Network, ”HTML: HyperText Markup Language,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML. [Använd 1 November 2026].

[2] World Wide Web Consortium, ”Accessibility | Our mission,” [Online]. Available: https://www.w3.org/mission/accessibility/. [Använd 1 November 2026].

[3] Figma, Inc, ”What is User Experience (UX) Design?,” [Online]. Available: https://www.figma.com/resource-library/what-is-ux-design/. [Använd 1 November 2026].

[4] Yale University, ”Prototypes,” [Online]. Available: https://usability.yale.edu/ux/build/design-documentation/prototypes. [Använd 1 November 2026].

[5] C. Coyier, ”A Complete Guide to CSS Flexbox,” 28 May 2026. [Online]. Available: https://css-tricks.com/snippets/css/a-guide-to-flexbox/. [Använd 1 November 2026].

[6] M. Berezhna, ”Separation of Concerns: A Fundamental Principle in Software Development,” 11 March 2025. [Online]. Available: https://dev.to/marie_berezhna/separation-of-concerns-a-fundamental-principle-in-software-development-oaj. [Använd 1 November 2026].

[7] Mozilla Developer Network, ”<iframe> HTML inline frame element,” [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/iframe. [Använd 1 November 2026].